

Maze Game Solvers

30 APR 2025 - 01 MAY 2025

Objective

Create a playable, standalone maze game and use OOP to visualise algorithmic solvers. To visually demonstrate behaviours of different algorithms whilst practicing implementation. Use AI to increase the efficiency of implementation as much as possible.

Scheme

OOP

1. Although at a small scale, it is important to recognise the importance of OOP and design patterns for a more cohesive and extensible program.
2. I will use strategy pattern to showcase multiple maze solver strategies

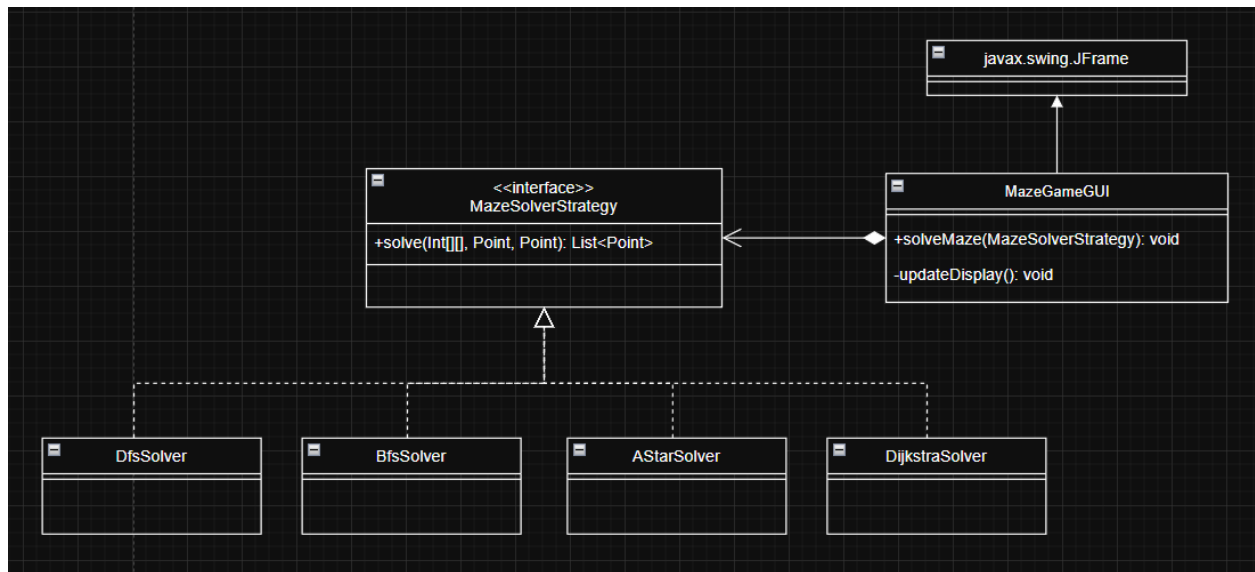
Solving Algorithms

1. There are many types of algorithms, such as brute force, divide and conquer, dynamic programming, etc. I will practice and explore some use cases, strengths and weaknesses.
2. The chosen algorithms are DFS, BFS, A* and Dijkstra's algorithm.
3. These are one of the most common algorithms for graph traversal.

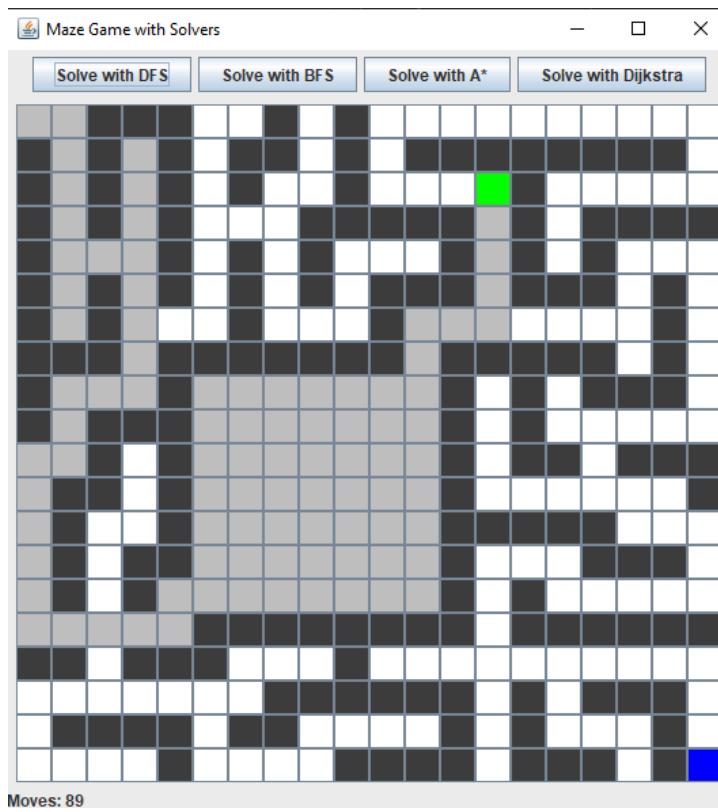
Notes

- The chosen algorithms aren't meant to be optimal for maze solving but will reveal the behaviours of each.
- The maze tiles all have the same "cost" of traversal per "step", so the more computationally efficient algorithms may not necessarily yield better results.

UML Class Diagram



GUI



Specifications

- GUI created using javax.swing
- Manually playable using arrow keys or alternatively playable in console entering WASD (separate main method)
- GUI includes solver visualisers, showing tiles visited (process), and moves count (cost)
- Strategy pattern for implementing solvers
- Black tiles represent walls, white denotes open space, blue denotes the end, green denotes the player, grey denotes visited tile.
- Run via jar file located in artifacts/MazeGame_jar/MazeGame.jar

Results

- DFS searches tree branch to tree branch, with a LIFO approach, checking every dead end before turning back.
- BFS searches the tree level by level, with a FIFO approach, almost like a pool of water would flood the maze from entry to finish.
- A* uses heuristics, in this case, the direction to the finish is known, so it will prioritise paths that yield shorter distances to the finish before backtracking.
- Dijkstra's algorithm is not designed for equal cost graphs with tile-like edges such as our maze. It is a greedy algorithm that chooses the cheapest/shortest path based on known information. In our maze scenario, it should behave very similarly to BFS. It will only differ due to queue priority as opposed to FIFO.

These results are visually identifiable using the program, and can be a reference as for how different solving algorithms 'think'.